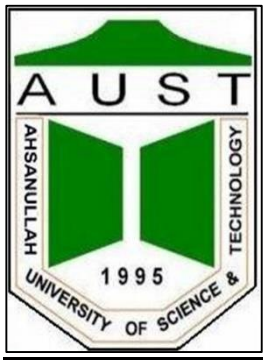
AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2103
Course Title: Data Structures Lab

Assignment No: 05

Date of Submission: 20/2/2026
Submitted by,
Name: Durjoy Roy
Student ID: 0072410101055
Lab Section: B1
Academic Semester: Spring 2025
Program: BSc in CSE

Question:

Implement the below functions of doubly linked list in a single .cpp file.

(1) insert first

(2) insert last

(3) insert anywhere between two nodes (by position, by value)

(4) delete first

(5) delete last

(6) delete from anywhere between two nodes (by position, by value)

(7) printingF() and printingB() -> functions to print the linked list

(8) searching() -> function to search a value in the linked list

(9) last_node() -> function to print the value of the last node

(10) previous_of_last_node() -> function to print the value of the previous node of

last node

(11) list_size() -> function to print the size of the linked list

(12) reversePrint() -> function to print the linked list in reverse order

(13) main function -> no manual node creation/connection is allowed

Solution:

```
#include<bits/stdc++.h>
using namespace std;

struct node
{
   int data;
   node* pre;
   node* next;
};

node* head = NULL;
```

```
node* tail = NULL;

void insertFirst(int val)
{
   node* temp = new node();
   temp->data = val;
   temp->pre = NULL;
   temp->next = head;

   if(head == NULL)
   {
      head = tail = temp;
   }
   else
   {
      head->pre = temp;
      head = temp;
   }
}

void insertLast(int val)
{
   node* temp = new node();
   temp->data = val;
```

```
    temp->next = NULL;
    temp->pre = tail;

    if(head == NULL)
    {
        head = tail = temp;
    }
    else
    {
        tail->next = temp;
        tail = temp;
    }
}

void insertByPosition(int pos,int val)
{
    if(pos == 1)
    {
        insertFirst(val);
        return;
    }

    node* curr = head;
    for(int i=1; i<pos-1 && curr!=NULL; i++)
```

```
    {
        curr = curr->next;
    }

    if(curr == NULL || curr == tail){
        insertLast(val);
        return;
    }

    node* temp = new node();
    temp->data = val;
    temp->next = curr->next;
    temp->pre = curr;
    curr->next->pre = temp;
    curr->next = temp;
}

void insertByValu(int val1,int val2)
{
    node* curr = head;
    while(curr!=NULL && curr->data != val1)
    {
        curr = curr->next;
    }
```

```
    if(curr == NULL) return;
    if(curr == tail){
        insertLast(val2);
        return;
    }

    node* temp = new node();
    temp->data = val2;
    temp->next = curr->next;
    temp->pre = curr;
    curr->next->pre = temp;
    curr->next = temp;
}

void deleteFirst(){
    if(head == NULL) return;
    if(head == tail){
        head = tail = NULL;
    }else{
        head = head->next;
        head->pre = NULL;
    }
}
```

```
void deleteLast(){
   if(tail == NULL) return;
   if(head == tail){
      head = tail = NULL;
   }else{
      tail = tail->pre;
      tail->next = NULL;
   }
}

void deleteByPosition(int pos){
   if(pos == 1){
      deleteFirst();
      return;
   }
   node* curr = head;
   for(int i=1;i<pos && curr!=NULL;i++){
      curr = curr->next;
   }
   if(curr == NULL) return;
   if(curr == tail){
      deleteLast();
      return;
```

```
    }
    curr->pre->next = curr->next;
    curr->next->pre = curr->pre;
}

void deleteByValu(int val){
     node* curr = head;
    while(curr!=NULL && curr->data != val)
    {
        curr = curr->next;
    }
    if(curr == NULL) return;
    if(curr == head){
        deleteFirst();
        return;
    }
    if(curr == tail){
        deleteLast();
        return;
    }
    curr->pre->next = curr->next;
    curr->next->pre = curr->pre;
}
```

```
void printingF()
{
   node* curr = head;
   while(curr != NULL)
   {
      cout << curr->data << " ";
      curr = curr->next;
   }
   cout << endl;
}

void printingB(){
   node* curr = tail;
   while(curr != NULL){
      cout << curr->data << " ";
      curr = curr->pre;
   }
   cout << endl;
}

void searching(int val){
   node* curr = head;
   int pos = 1;
   while(curr!=NULL){
```

```
        if(curr->data == val){
            cout << "Found at position: " << pos << endl;
            return;
        }
        curr = curr->next;
        pos++;
    }
    cout << "Not Found" << endl;
}

void last_node(){
    if(tail!=NULL){
        cout << "Last Node: " << tail->data << endl;
    }
}

void previous_of_last_node(){
     if(tail!=NULL && tail->pre!=NULL){
        cout << "Previous of Last Node: " << tail->pre->data << endl;
    }
}

void list_size(){
    node* curr = head;
```

```
    int s = 0;
    while(curr !=NULL){
        s++;
        curr = curr->next;
    }
    cout << "Size: " << s << endl;
}

void reversePrint(){
    printingB();
}

int main()
{
    insertFirst(10);
    insertFirst(5);

    insertLast(20);
    insertLast(30);

    insertByPosition(3,15);
    insertByValu(20,25);
    insertByValu(30,35);
```

```
    printingF();
    printingB();

    deleteFirst();
    deleteLast();
    deleteByPosition(4);
    deleteByValu(10);
    printingF();

    searching(10);
    searching(20);

    last_node();
    previous_of_last_node();
    list_size();
    reversePrint();
}
```

Output:

```
5 10 15 20 25 30 35
35 30 25 20 15 10 5
15 20 30
Not Found
Found at position: 2
```

Last Node: 30

Previous of Last Node: 20

Size: 3

30 20 15